

Certified frontiers for tensor-network contraction ordering

Jinguo Liu

Advanced Materials Thrust, The Hong Kong University of Science and Technology
(Guangzhou)

July 23, 2026

Abstract

Contraction-order optimizers for tensor networks are routinely compared against each other, but almost never against certified references, so it is unknown whether reported improvements approach any fundamental limit. We carry out such a study on two closed benchmark networks—a 250-tensor random 3-regular graph and a 561-tensor Sycamore-like random-circuit proxy—under a fixed protocol: 90-second single-threaded budgets, independent rescoring of every contraction tree from its topology, and adversarial validation. We report three results. First, the single largest practical lever we find is a mis-set default: annealing toward an aspirationally low memory target ($sc_{\text{target}} = 20$) costs 4.3 and 9.7 in $\log_2$ flops on the two instances ($20\times$ and $800\times$), and simply removing the target recovers the entire loss; thirteen mechanically distinct search mechanisms, implemented and scored under the same protocol, recover nothing further. Second, the resulting frontier is reproduced independently by cotengra’s simulated-annealing refiner, while its hyper-optimization mode remains 1.7–2.4 behind at equal budget; the frontier is also budget-independent over a $10\times$ range. Third, we localize the optimum: on the circuit proxy the frontier is provably width-optimal and the true minimum lies in $[53, 61.544]$; we prove two sum-form lower bounds on the log-sum contraction cost and show that they, and every bound computable from the isoperimetric profile alone, are capped at the bisection width—an impossibility result explained by an explicit low-boundary cut certificate and by a measured conservation-like behavior of frontier cost profiles. All claims carry machine-checkable certificates or full audit trails from the 31-attempt autonomous research loop that produced them, which caught three distinct illusory-gain modes along the way.

1 Introduction

The cost of contracting a tensor network is governed by the order in which its tensors are combined. Finding a good order is the computational bottleneck of classical random-circuit simulation [7, 6], and a rich ecosystem of optimizers has grown around the problem: simulated-annealing refiners [5], hyper-optimized portfolios over greedy and hypergraph-partitioning constructions [3, 10], treewidth machinery imported from the parameterized-algorithms community [2, 12], and exact methods for small networks [9]. These tools are compared against each other, on wall-clock budgets, and the field’s implicit quality standard is relative: an optimizer is good if it beats another optimizer.

This paper asks the absolute questions. On fixed benchmark instances and a fixed time budget: (i) how much of the attainable quality do shipped defaults throw away; (ii) where do all known search paradigms actually converge; and (iii) how far is that convergence point from the true optimum—and can the gap be certified rather than conjectured?

We answer all three on two closed networks under a hardened evaluation protocol (Sec. 2). The answers are summarized in Figs. 1–3: a mis-set default memory target dominates every algorithmic effect we could produce (Sec. 3); thirteen search mechanisms and two independent toolchains converge to a single frontier (Sec. 4); and that frontier is provably width-optimal, with the optimum localized to a certified interval whose residual resists an entire family of lower-bound techniques for a reason we prove (Sec. 5). We close with the behavior of the problem at larger scale (Sec. 6) and with the autonomous-loop methodology that produced—and repeatedly corrected—these claims (Sec. 7).

2 Setup and evaluation protocol

Instances and cost model. A closed tensor network is a graph $G = (V, E)$: vertices are tensors, edges are summed indices, and every index here has dimension 2. A contraction order is a full binary tree whose leaves are the tensors; each internal node v contracts its two children into an intermediate tensor. Writing cost_v for the $\log_2$ floating-point cost of node v (the sum of $\log_2$ dimensions over the union of the operands’ indices), the two standard quality measures are the total time cost

$$\text{tc} = \log_2 \sum_v 2^{\text{cost}_v}, \quad (1)$$

a log-sum-exp over all $|V| - 1$ contractions, and the space cost sc , the $\log_2$ size of the largest intermediate. We use two instances throughout: *reg3_250*, a random 3-regular graph with 250 tensors and 372 indices, and *sycamore_m20*, a deterministic Sycamore-scale random-quantum-circuit proxy with 561 tensors and 963 indices (53 qubits on an 8×7 grid minus three corners, 20 cycles of two-qubit gates in the standard four-coloring pattern, contracted to a single amplitude). We state explicitly that the latter is a structurally faithful proxy, not the physical Sycamore circuit.

Protocol. Every optimizer run gets 90 seconds of single-threaded wall clock per instance. Emitted trees are rescored by an independent checker that recomputes tc and sc from the tree topology and the original index sets alone; optimizer-reported numbers are never trusted. Instances are randomly relabeled per run, so memorized answers keyed on input encoding do not transfer. A best-known value (“record”) only moves when a run beats it and a second run under fresh relabeling confirms; the worse of the two runs is recorded. CPU-time accounting rejects hidden parallelism. Reference rows are measured best-of-three with the identical pipeline. The full validation harness, including its negative-control self-tests, is described in Sec. 7.

3 A mis-set default dominates all algorithmic effects

The TreeSA-family annealers [5] optimize a composite objective in which the space cost is pulled toward a target $\text{sc}_{\text{target}}$, with a penalty active whenever $\text{sc} > \text{sc}_{\text{target}}$. The reference implementation we study (omeco, mirroring OMEinsumContractionOrders.jl) ships $\text{sc}_{\text{target}} = 20$. On instances whose good trees have naturally larger intermediates—the optimum-quality trees here have $\text{sc} = 34$ and 53—this default is not merely unhelpful. It actively drags the search into a low-memory, high-flop region of tree space.

Figure 1 shows the resulting tc at a fixed 90-second budget as a function of $\text{sc}_{\text{target}}$. **This figure is our first main result**, and the transition it reveals is a cliff, not a slope: every target below the natural space cost of good trees is roughly equally harmful (medians 45.6–47.0 on *reg3_250*, 71–77 on *sycamore_m20*), quality snaps to the frontier exactly at the natural value ($\text{sc}_{\text{target}} = 34$ and 53), and stays there unchanged through $\text{sc}_{\text{target}} = \infty$, i.e., with the memory objective deleted. The harm is strictly one-directional. Two finer observations: targets just *below* the natural value are the worst on *sycamore_m20* (median 77.0 at $\text{sc}_{\text{target}} = 50$, versus 72.1 at the shipped default of 20), consistent with the penalty being most active precisely near the boundary it cannot reach; and the natural-value target itself slightly outperforms $\text{sc}_{\text{target}} = \infty$ there (61.51 versus 61.67, non-overlapping ranges over three runs), so a correctly set finite target is never worse than none. At the shipped default the damage is 4.3 $\log_2$ units on *reg3_250* and 9.7 on *sycamore_m20*: a $20\times$ and $800\times$ flop overhead, recovered by a one-line change. An ablation within one of our search mechanisms confirms the attribution: holding everything else fixed and moving the penalty target to the natural space cost moved *reg3_250* from $\text{tc} = 56.8$ to 40.0, while that mechanism’s novel component contributed 0.002.

The practical rule is: *never anneal toward a memory target below the natural space cost of good trees; absent a physical memory budget, use the natural value or no target at all*. The corresponding fix (default $\text{sc}_{\text{target}} = \infty$, with finite values only when a real memory budget is supplied) applies to omeco and, we expect, upstream implementations sharing the default.

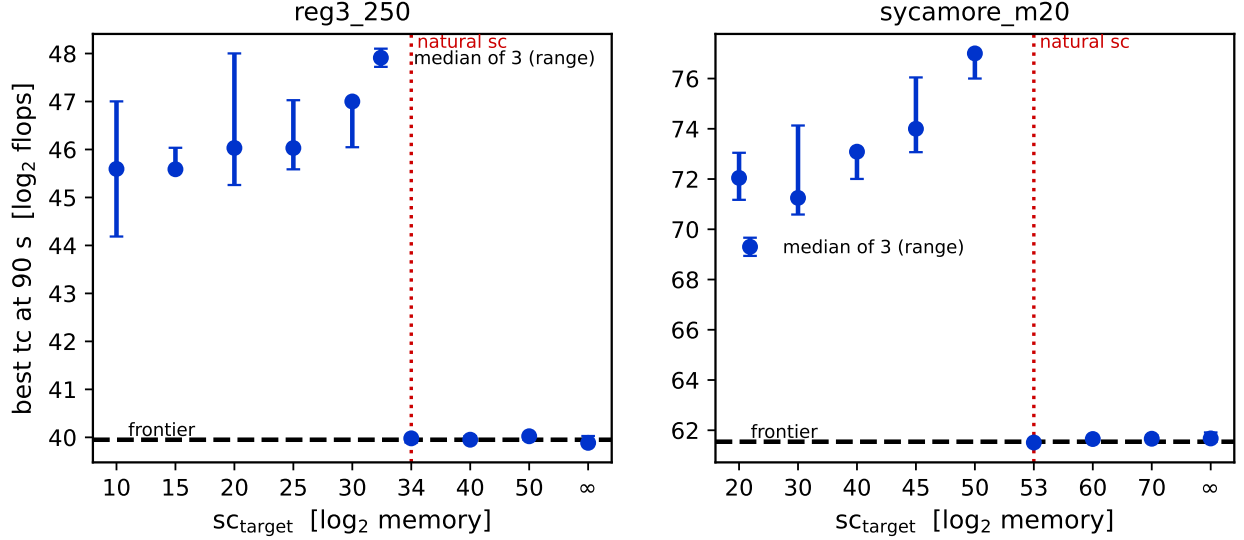


Figure 1: Best tc reached in 90s of single-threaded annealing versus the memory target sc_{target} (median of three runs; whiskers span the range). The dashed line marks the frontier value of Sec. 4; the dotted vertical line marks the natural space cost of optimum-quality trees. The transition is a cliff at the natural value: every smaller target is comparably harmful, and every target at or above it, including $sc_{\text{target}} = \infty$, reaches the frontier. The shipped default is $sc_{\text{target}} = 20$.

4 Thirteen mechanisms, two toolchains, one frontier

With the objective corrected, we attempted to beat the resulting reference with mechanically distinct search paradigms, implemented independently and scored under the protocol of Sec. 2: coarse subtree-transfer moves and basin hopping [1], evolutionary replica populations, exact-penalty ramps through infeasible regions, tabu search with strategic oscillation, large-neighborhood destroy-and-repair, beam search with lookahead, Boltzmann-greedy portfolios [3, 8], recursive balanced min-cut construction [11, 10], elimination-order annealing [2], order-then-tree interval dynamic programming [4], hierarchical coarsen-then-exact solves, structure-guided sweeps exploiting the circuit’s spacetime geometry, and profile-aware annealing (Sec. 5).

Figure 2 shows every scored result as its distance from the best verified value. All thirteen mechanisms land within $0.35 \log_2$ of the corrected reference on both instances—most within noise—and none set a confirmed record against it. The same frontier is reproduced by an independent toolchain: cotengra’s simulated-annealing refiner [3] ties it to within 0.05 and 0.14, while cotengra’s hyper-optimization mode alone (greedy and hypergraph partitioning portfolios, without SA refinement) remains 1.7 and 2.4 behind at the same matched budget. The frontier is also budget-independent: extending the budget tenfold, to 900 seconds, moves it by less than 0.1 (Fig. 6a). We conclude that at this scale the frontier is a property of the cost landscape, not of any particular search mechanism, schedule, or implementation.

Two honest qualifications. First, these comparisons are matched-budget, single-threaded, and slicing-free by design; cotengra’s strengths in massively parallel trial sampling and sliced contraction lie outside this protocol. Second, convergence of many searchers is evidence of a shared basin, not of optimality—which is precisely why certified bounds are needed, and to which we now turn.

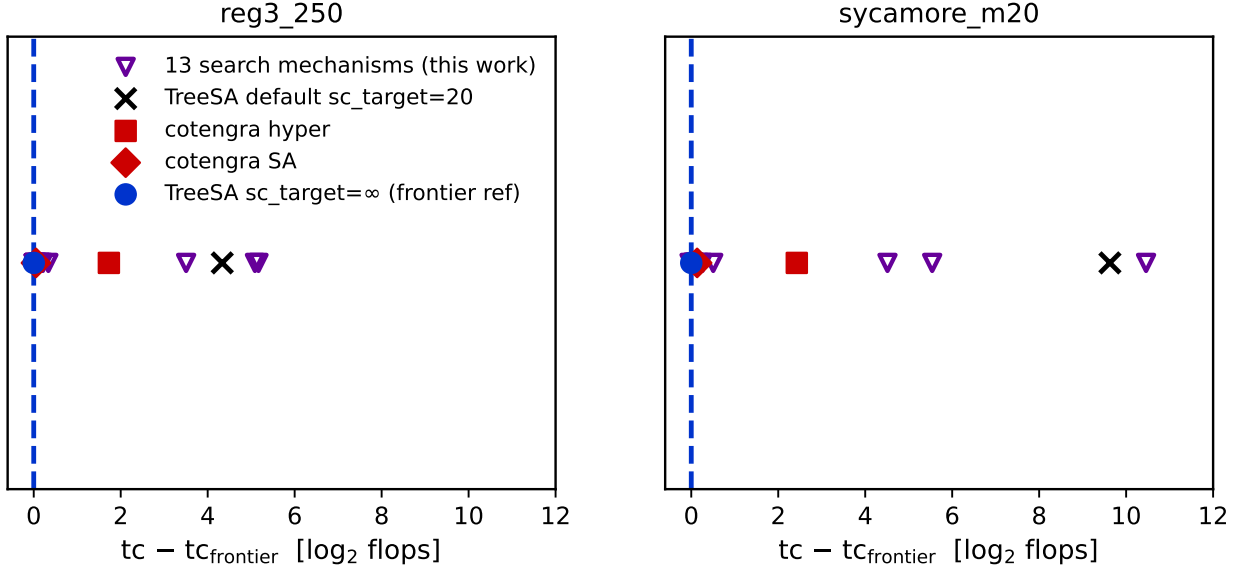


Figure 2: Distance of every scored optimizer from the frontier value (dashed line at zero). Open triangles: the thirteen search mechanisms of this work. Filled symbols: the corrected TreeSA reference ($sc_{\text{target}} = \infty$, circle), cotengra’s SA refiner (diamond), cotengra’s hyper-optimizer (square), and the shipped-default TreeSA (cross).

5 Certified bounds: width-optimality, the interval, and an impossibility result

5.1 Max-form bounds and width-optimality of the frontier

Every classical lower-bound technique for contraction cost is *max-form*: it bounds the single largest contraction. The Markov–Shi correspondence [7] equates the minimal largest contraction rank with the treewidth of the line graph $L(G)$, and any balanced edge of a contraction tree yields a cut whose boundary lower-bounds the largest contraction.

On `sycamore_m20` these tools give a sharp result. An explicit temporal cut severing all 53 qubit world-lines gives a balanced cut of value 53; extensive search finds nothing smaller, and the frontier trees achieve $sc = 53$ exactly. The frontier is therefore *width-optimal*: no contraction tree of this network has a smaller largest intermediate, and consequently $tw(L(G)) = 52$. Combined with the frontier value, **our second main result** is the certified localization

$$tc_{\min}(\text{sycamore_m20}) \in [53, 61.544]. \quad (2)$$

On `reg3_250`, certifying the analogous statement is obstructed by the known hardness of certifying high treewidth on expanders: the best machine-verified certificate (a 32-vertex minor of $L(G)$, checked by two independent exact solvers) gives $tw(L(G)) \geq 18$ against a constructive upper bound of 34, so the structural picture $39.95 \approx 34 + (\log\text{-sum overhead})$ remains uncertified.

The width bound, however, has a ceiling: tc is a log-sum (1) over $\sim |V|$ contractions, and the frontier’s excess over the width bound—8.5 on `sycamore_m20`, close to the maximal possible $\log_2 561 = 9.1$ —is invisible to any bound on a single contraction. Closing the interval (2) from below requires *sum-form* bounds.

5.2 Sum-form bounds and their collapse

For any tree, each internal node v with leaf set S_v satisfies $\text{cost}_v \geq |\partial S_v|$, the boundary of S_v in G . Writing $b(k)$ for the isoperimetric profile of G (the minimal boundary over all k -vertex subsets), we prove:

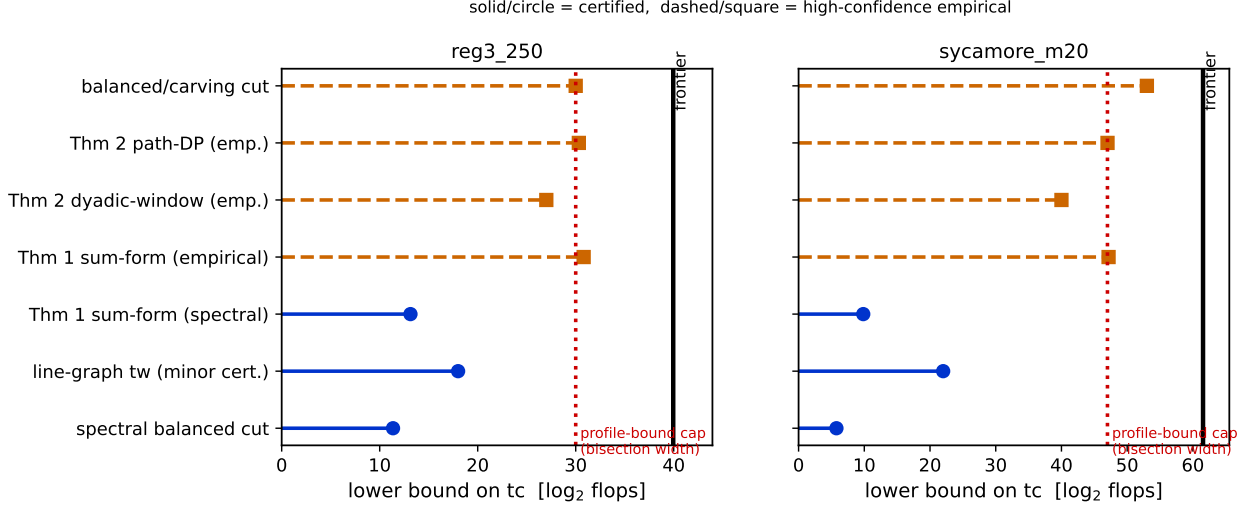


Figure 3: Lower-bound ladder for both instances. Solid stems with circles are certified (machine-checkable certificates); dashed stems with squares are high-confidence empirical values (explicit cuts found by search; searching harder can only lower them). Every profile-based bound (Theorems 1 and 2) is capped at the bisection width (dotted line, Observation 1); only the balanced/carving cut exceeds it. The solid vertical line is the optimizer frontier.

Theorem 1 (Sum-form profile bound). *For every contraction tree, $tc \geq \log_2 F(n)$ where F obeys the recursion $F(1) = 0$, $F(k) = 2^{b(k)} + \min_{1 \leq j \leq k/2} [F(j) + F(k-j)]$.*

Theorem 2 (Dyadic-window bound). *Descending from the root always into the larger child visits, for every dyadic window $W_j = (n/2^{j+1}, n/2^j]$, at least one internal node whose leaf-set size lies in W_j ; these nodes are distinct, hence $tc \geq \log_2 \sum_j 2^{\min_{k \in W_j} b(k)}$.*

Both theorems are validated computationally (exhaustively at small n , and on 3000 random trees). Both fail to beat the max-form cap (Fig. 3): on `sycamore_m20` they reach 47.2 and 47.0 against the carving value 53. The reason is structural, and we state it as **our third main result**:

Observation 1 (Profile bounds are capped at the bisection width). *Any lower bound computed from the isoperimetric profile $b(\cdot)$ alone is at most $\max_k b(k) + \log_2 \log_2 n$, and $\max_k b(k)$ is the bisection width—strictly below the balanced-cut (carving) value whenever the profile dips off-center. On `sycamore_m20` it does: we exhibit a triply-verified 141-tensor subset with boundary exactly 40, against temporal-slab boundaries of 53 (Fig. 4).*

The mechanism is twofold: the log-sum-exp collapses a sum of window terms to within $\log_2 J$ of its largest term, and the relaxation from nested tree cuts to per-size minima discards exactly the joint structure (carving-type constraints) that forces many simultaneous large cuts. The $47 \rightarrow 53$ gap lives in that joint structure; the $53 \rightarrow 61.5$ gap is the log-sum residual. Certifying either further requires bound machinery of carving/branch-width type, which we leave open.

A complementary measurement explains why search cannot exploit the residual either. Comparing frontier trees of equal tc but different peak widths (Fig. 5), reducing the largest contraction by one unit systematically multiplies the number of contractions just below the peak—total flops behave as if conserved under profile reshaping. Profile-aware annealing (sharpened energies, peak-targeted proposals) produces trees statistically indistinguishable from plain annealing, consistent with the frontier profile being at a constrained optimum.

6 Where scale changes the game

At 250–561 tensors the frontier is budget-independent and mechanism-independent; at 1000–1238 tensors neither holds (Fig. 6). Ninety seconds no longer suffices for convergence, single-run variance

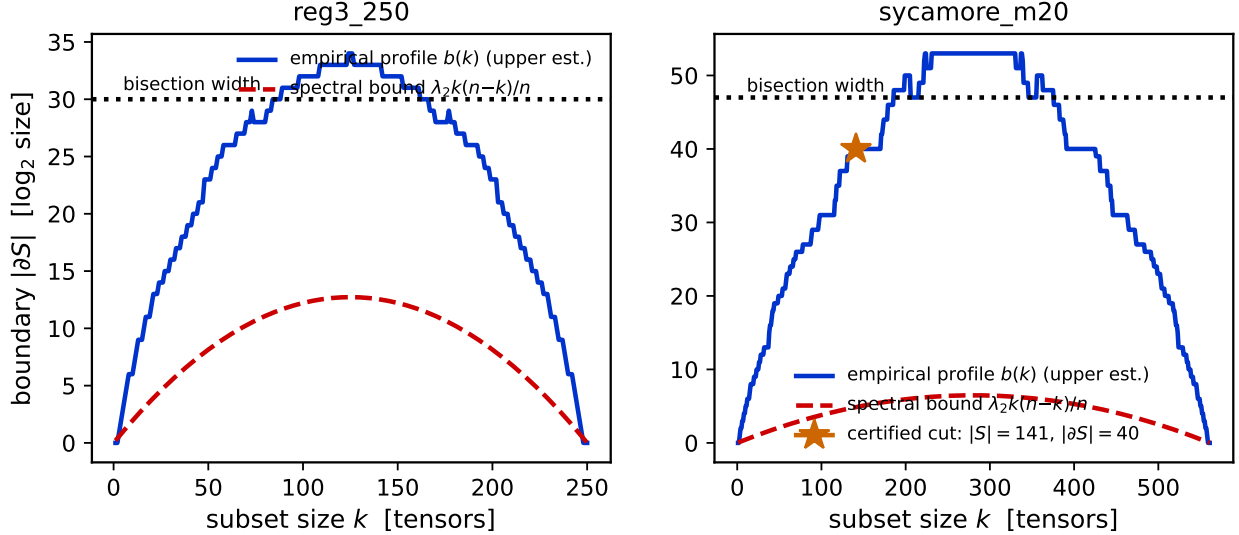


Figure 4: Isoperimetric profiles. The empirical profile $b(k)$ (solid) dips well below the bisection width (dotted) at off-center sizes; the certified star marks the 141-tensor sycamore_m20 subset with boundary exactly 40. The spectral bound (dashed) is certified but loose. This dip is what caps all profile-based bounds (Observation 1).

grows to $1\text{--}5 \log_2$ units—exceeding every genuine method difference we measured—and schedule design (continuous ramps versus restart ladders) trades early against late quality. Our confirmation-based record protocol correctly refuses lucky records in this regime, but symmetrically prevents real sub-variance improvements from registering; meaningful optimizer comparisons at scale therefore require distributional reporting (medians over repeated runs), as practiced in [3], and we flag our single-run scale numbers as indicative only.

7 Methodology: a research loop that distrusts itself

The results above were produced by an autonomous research loop—31 optimizer attempts over six cycles, each in an isolated worktree with a pre-registered hypothesis, scored by a sandboxed validator—and the loop’s central design lesson is that its most important outputs were *retractions* (Fig. 7). Three failure modes were caught by the harness rather than by luck:

(i) *Schedule exploitation.* The first gate (a speed-versus-baseline bar) was passed by budget hygiene alone—anytime writes and early termination—with no algorithmic content. The fix was structural: fold the exploit into the reference, making schedule-only candidates score zero by construction, and keep the exploit as a permanent negative control.

(ii) *Mis-tuned-baseline gains.* Eight mechanisms then “beat” the references by up to $800\times$ —until a strengthened reference (the one-line $\text{sc}_{\text{target}}$ fix of Sec. 3) reclaimed the entire gain. What survived was the tuning discovery itself, and the leaderboard practice of ratcheting references to absorb every discovered improvement, so that only genuinely new effects can score.

(iii) *Instrument bias.* A failed attempt’s diagnosis revealed that the independent scorer silently rejected contraction trees deeper than ~ 475 (a recursion limit), biasing the admissible space against exactly the deep, MPS-like trees that suit circuit instances. The scorer was fixed, its negative-control suite re-passed, and the affected hypothesis re-tested.

The pattern that generalizes: score only through a hardened, independent validator (recompute everything, trust nothing the candidate reports); maintain executable negative controls, and when a new exploit is found, patch the harness and add the exploit to the control suite; ratchet references so yesterday’s wins are tomorrow’s baselines; and treat certified bounds, not incumbent-versus-incumbent deltas, as the quality standard. Under this regime, every claim in this paper is either

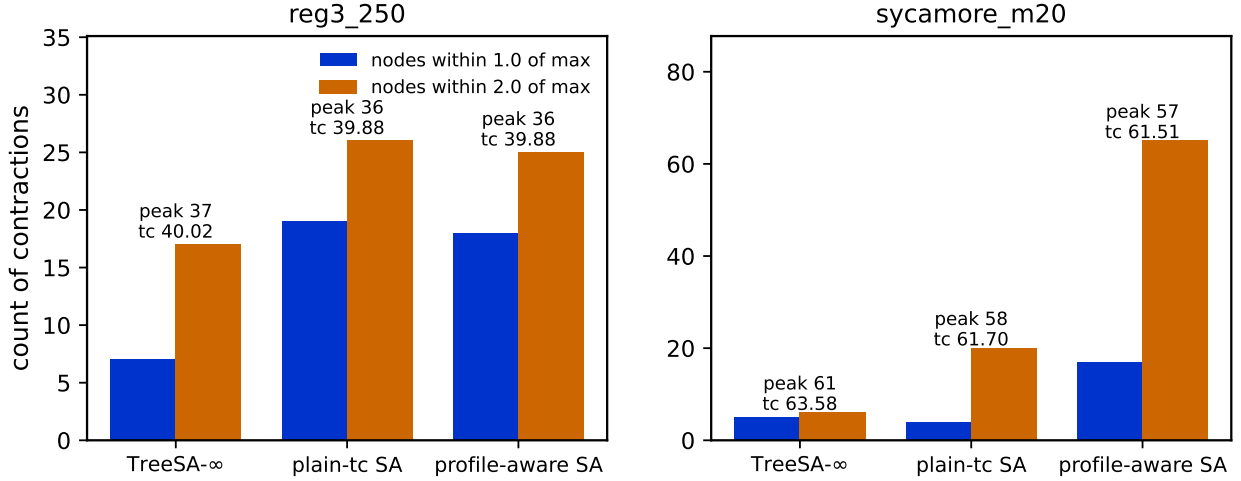


Figure 5: Cost-profile conservation at the frontier. Bars count contractions within 1.0 (solid) and 2.0 (hatched) of the largest; boxes give each tree’s peak cost and tc. Trees that lower the peak by one unit acquire several times more near-peak contractions at essentially unchanged tc.

machine-certified, or labeled high-confidence with its evidence, or was retracted before publication—and the audit trail (per-attempt logs, certificates, and cycle reports) ships with the code.

8 Conclusions

On standard contraction benchmarks under a fixed matched-budget protocol, we found that (1) the dominant practical effect is a shipped default: an aspirational memory target costs $20\times$ – $800\times$ in flops, is harmful in one direction only, and is fixed by removing the target unless a physical memory budget exists; (2) with that fixed, thirteen search mechanisms and two independent toolchains converge on a single, budget-independent frontier that simulated-annealing refinement—from either toolchain—reaches and hyper-optimization alone does not; and (3) on the circuit proxy the frontier is certifiably width-optimal, the optimum lies in $[53, 61.544]$, and no bound computable from the isoperimetric profile can close the interval further (Observation 1)—the remaining gap is jointly-nested-cut structure plus an apparently conserved log-sum residual.

The implications we would highlight: defaults in shipped optimizers deserve adversarial benchmarking, because a parameter can outweigh a decade of mechanism design; certified references, even partial ones, change what optimizer comparisons mean; and carving/branch-width-type machinery is the right target for future bound development, as profile methods are provably exhausted. Natural next steps are carving-width lower-bound certification, variance-aware protocols for large instances, physical circuit instances in place of proxies, and heterogeneous bond dimensions, where the weighted problem is largely unexplored.

Code and data availability

The optimizer (omeco), the validation harness, all certificates (treewidth minors, cut sets, and their verification scripts), per-attempt logs, and the cycle reports of the research loop are available in the project repository.

Acknowledgments

The research loop was orchestrated with LLM-based agents under pre-registered hypotheses and independent validation; all certified claims were verified by machine-checkable artifacts independent

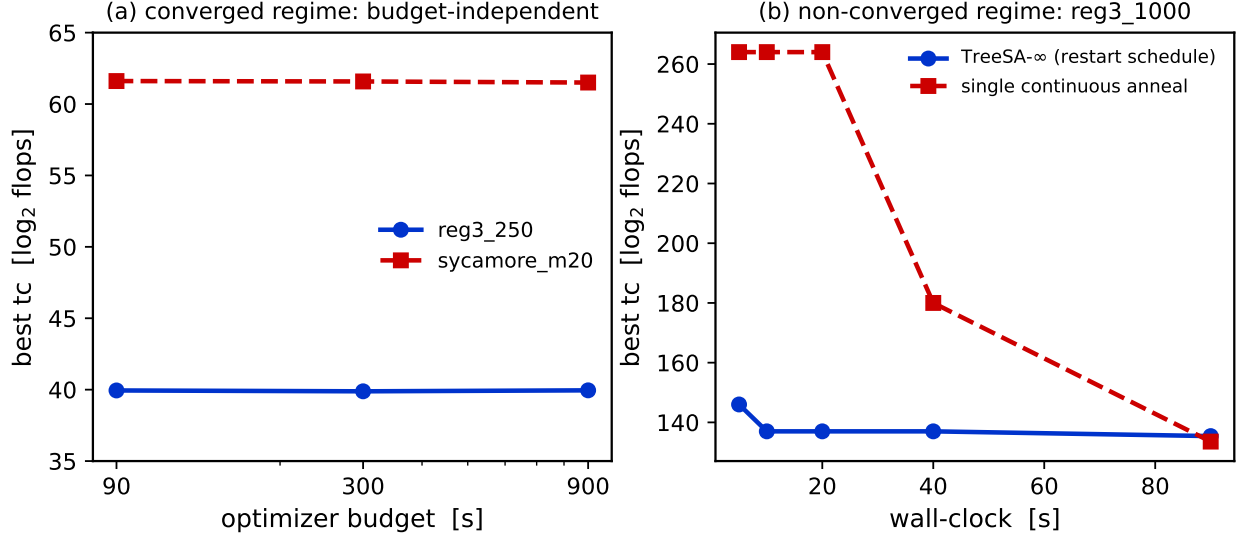


Figure 6: (a) Converged regime: the frontier on both benchmark instances is unchanged from 90 to 900 seconds. (b) Non-converged regime (reg3_1000): anytime curves of the restart-ladder reference versus a single continuous anneal; early and late quality trade off, and run-to-run variance (not shown) exceeds the differences.

of any agent output.

References

- [1] Optimizing tensor network partitioning using simulated annealing. 2025.
- [2] E. Dumitrescu, A. L. Fisher, T. Goodrich, T. Humble, Blair D. Sullivan, and Andrew Wright. Benchmarking treewidth as a practical component of tensor network simulations. *PLoS ONE*, 13, 2018.
- [3] Johnnie Gray and S. Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2020.
- [4] Cameron Ibrahim, Danylo Lykov, Zichang He, Y. Alexeev, and Ilya Safro. Constructing optimal contraction trees for tensor network quantum circuit simulation. *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8, 2022.
- [5] G. Kalachev, Pavel Panteleev, and M. Yung. Multi-tensor contraction for xeb verification of quantum circuits. 2021.
- [6] Yong Liu, Yaojian Chen, Chu Guo, Jiawei Song, X. Shi, Lin Gan, Wenzhao Wu, Wei Wu, H. Fu, Xin Liu, Dexun Chen, Zhifeng Zhao, Guangwen Yang, and Jiangang Gao. Verifying quantum advantage experiments with multiple amplitude tensor network contraction. *Physical review letters*, 132 3:030601, 2022.
- [7] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [8] Sheela Orgler and Mark Blacher. Optimizing tensor contraction paths: A greedy algorithm approach with improved cost functions. *ArXiv*, abs/2405.09644, 2024.
- [9] R. Pfeifer, J. Haegeman, and F. Verstraete. Faster identification of optimal contraction sequences for tensor networks. *Physical review. E, Statistical, nonlinear, and soft matter physics*, 90 3:033315, 2013.

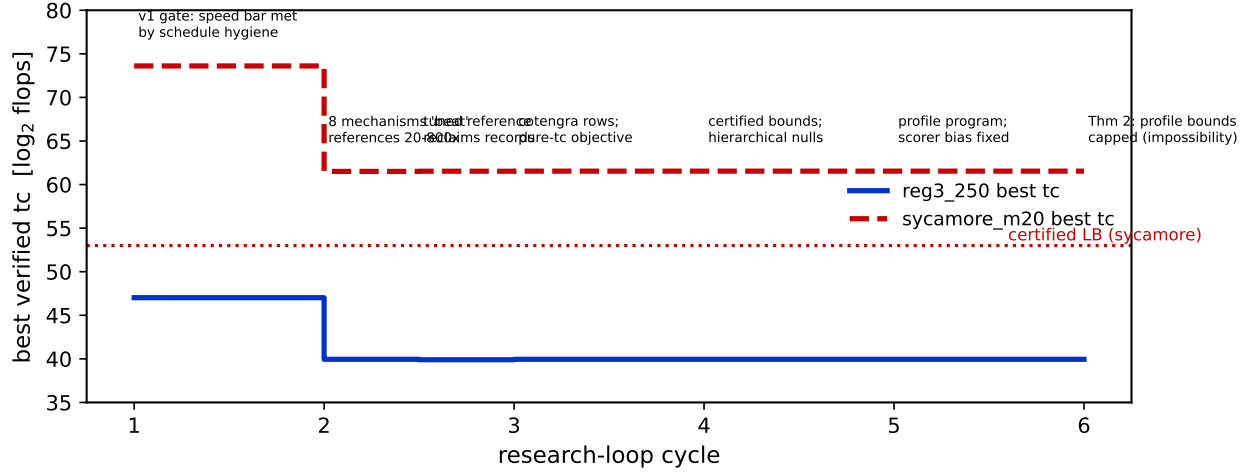


Figure 7: Best verified tc over the six research-loop cycles. Apparent progress in cycle 2 is reclaimed by the strengthened reference (cycle 2.5); thereafter the frontier is flat while the certified picture (bounds, impossibility) grows. Annotations mark the three instrument-caught failure modes.

- [10] Christoph Staudt, Mark Blacher, Julien Klaus, Farin Lippmann, and Joachim Giesen. Improved cut strategy for tensor network contraction orders. pages 27:1–27:19, 2024.
- [11] Ben Strasser. Computing tree decompositions with flowcutter: Pace 2017 submission. *ArXiv*, abs/1709.08949, 2017.
- [12] H. Tamaki. Positive-instance driven dynamic programming for treewidth. *Journal of Combinatorial Optimization*, 37:1283 – 1311, 2017.